

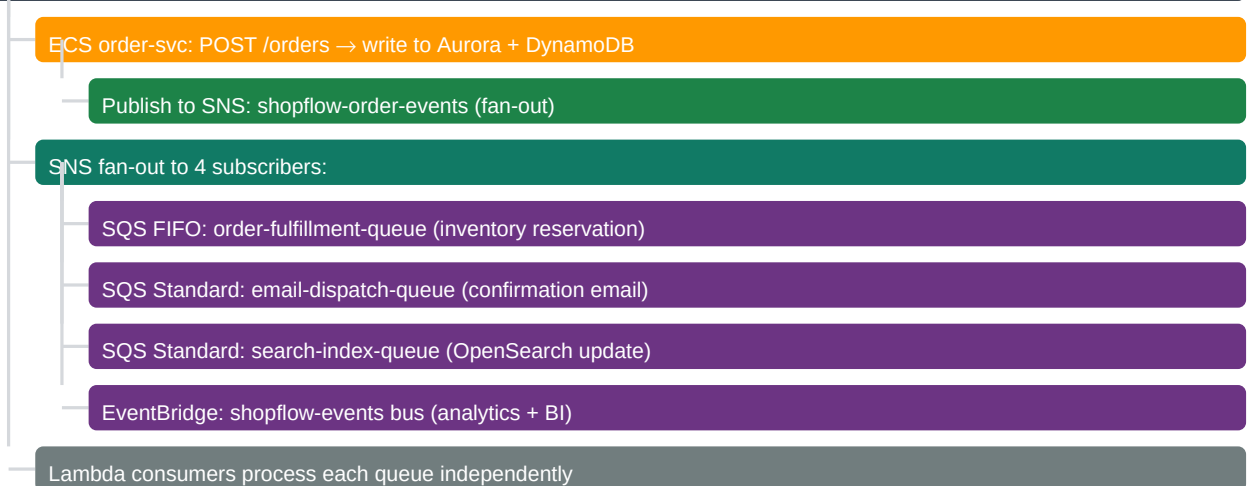
10-00

Chapter Overview — SQS, SNS & EventBridge at ShopFlow

ShopFlow uses three messaging services for different patterns: SQS for reliable point-to-point async processing (order fulfillment pipeline), SNS for fan-out notifications (order events to email + inventory + analytics), and EventBridge for event-driven integration between microservices and scheduled batch jobs.

Service	Pattern	ShopFlow Use-Case	Volume
SQS Standard	Point-to-point async	Email dispatch queue, image resize jobs	200K msg/day
SQS FIFO	Ordered, exactly-once	Order fulfillment pipeline (sequence matters)	50K msg/day
SNS Standard	Fan-out pub/sub	Order placed: broadcast to 4 subscribers	50K msg/day
SNS FIFO	Ordered fan-out	Inventory update to multiple warehouses	10K msg/day
EventBridge (default bus)	AWS service events	ECS task state changes, ECR scan results	5K events/day
EventBridge (custom bus)	Microservice events	shopflow.order.placed, shopflow.cart.updated	100K events/day
EventBridge Scheduler	Cron / one-time	Daily reports, cache warming, BF pre-warm	30 jobs/day

ShopFlow Order Processing — Full Async Pipeline



■ SNS + SQS fan-out pattern is the backbone of event-driven architecture on AWS. A single SNS Publish call delivers the event to all subscribers simultaneously. Each SQS subscriber processes at its own pace, applies its own retry policy, and can scale independently. If the email service is slow, it does not delay inventory reservation — queues decouple producers from consumers completely.

10-01

SQS Core Concepts — Queues, Messages & Visibility Timeout

SQS stores messages durably until a consumer reads and deletes them. Visibility timeout hides a message from other consumers while one consumer is processing it. If the consumer fails to delete within the timeout, the message reappears for retry. Dead-Letter Queue captures messages that fail `maxReceiveCount` times.

Concept	Definition	ShopFlow Config
Message retention	How long SQS stores unprocessed messages	14 days (max). Email queue: 3 days (email outdated after 3 days).
Visibility timeout	Duration message is hidden after ReceiveMessage	Order fulfillment: 5 min (inventory reservation SLA). Email: 30s.
maxReceiveCount	Times a message is delivered before DLQ	3 for email (transient failures). 5 for fulfillment (more retries).
Dead-Letter Queue (DLQ)	Receives messages after maxReceiveCount attempts	All queues have DLQ. DLQ retention: 14 days. Alert on DLQ depth >0.
Long polling	WaitTimeSeconds=20: waits for messages if none available	All Lambda ESM triggers use long polling (reduces empty polls).
Batch size	Messages per Lambda invocation (ESM)	Email: batch=10. Fulfillment: batch=1 (FIFO, must be ordered).

Email Dispatch Queue CDK

```
const emailDlq = new sqs.Queue(this,"EmailDlq",{
  queueName: "shopflow-email-dispatch-dlq",
  retentionPeriod: Duration.days(14),
});
const emailQueue = new sqs.Queue(this,"EmailQueue",{
  queueName: "shopflow-email-dispatch",
  visibilityTimeout: Duration.seconds(30),
  messageRetentionPeriod:Duration.days(3),
  deadLetterQueue: {
    queue: emailDlq,
    maxReceiveCount: 3,
  },
  encryption: sqs.QueueEncryption.KMS_MANAGED,
});
```

■ Set visibility timeout to at least 6× the Lambda function timeout. Lambda ESM uses long polling and locks the message for the duration of the batch processing. If Lambda timeout is 30s and visibility timeout is 30s, a slow invocation will cause the message to reappear before Lambda finishes — resulting in duplicate processing. For 30s Lambda: set visibility timeout to 3 minutes minimum.

10-02 SQS FIFO — Order Fulfillment with Exactly-Once Delivery

SQS FIFO guarantees exactly-once processing and strict ordering within a MessageGroupId. ShopFlow uses FIFO for the order fulfillment pipeline — inventory reservation and payment must process in order per order. MessageDeduplicationId prevents duplicate processing on producer retry.

FIFO Queue CDK

```
const fulfillmentDlq = new sqs.Queue(this,"FulfillmentDlq",{
  queueName: "shopflow-order-fulfillment-dlq.fifo",
  fifo: true,
  retentionPeriod: Duration.days(14),
});
const fulfillmentQueue = new sqs.Queue(this,"FulfillmentQueue",{
  queueName: "shopflow-order-fulfillment.fifo",
  fifo: true,
  contentBasedDeduplication: false, // we provide MessageDeduplicationId
  visibilityTimeout: Duration.minutes(5),
  deadLetterQueue: {
    queue: fulfillmentDlq,
    maxReceiveCount: 5,
  },
});
```

Send to FIFO Queue — Java

```
sqsClient.sendMessage(SendMessageRequest.builder()
  .queueUrl(fulfillmentQueueUrl)
```

```

.messageBody(json.writeValueAsString(orderEvent))
.messageGroupId(orderId) // FIFO: all messages for one order processed in order
.messageDeduplicationId(orderId + "#" + eventType + "#" + timestamp)
// Idempotent: retry with same deduplicationId = SQS deduplicates (5-min window)
.build();

```

FIFO Feature	Behavior	ShopFlow Application
MessageGroupId	Messages with same group ID processed in order	FIFO: all events for one order processed in sequence
MessageDeduplicationId	Exactly-once within 5-min deduplication window	orderId+eventType+timestamp: retry-safe publishing
Content-based deduplication	SHA-256 of body as dedup ID (auto)	Not used — explicit dedup ID more predictable
Throughput (no batch)	300 msg/s per queue (vs 300K for Standard)	ShopFlow fulfillment: 50K/day = 0.6 msg/s — well within limit
High-throughput mode	3,000 msg/s per queue	Enable if fulfillment throughput exceeds 300/s

■ SQS FIFO throughput limit: 300 messages/second per queue without batching, 3,000/s with SendMessageBatch (10 messages per batch). Black Friday peak: 10,000 orders/hour = 2.8 orders/s — well within limits. However, if order processing creates multiple FIFO messages per order (e.g., ORDER_PLACED + INVENTORY_RESERVED + PAYMENT_CAPTURED), multiply by 3× = 8.4 msg/s — still fine.

10-03

SNS Fan-Out — Order Events to Multiple Subscribers

SNS delivers one published message to all subscribed endpoints simultaneously. ShopFlow publishes every order event to SNS, which fans out to four SQS queues. Each SQS queue has independent consumers with different processing rates, retry policies, and scaling configurations.

SNS Topic + SQS Subscriptions CDK

```

const orderEventsTopic = new sns.Topic(this, "OrderEventsTopic", {
  topicName: "shopflow-order-events",
  displayName: "ShopFlow Order Events",
  masterKey: orderTopicKey,
});
// Subscribe 4 SQS queues to the SNS topic
[fulfillmentQueue, emailQueue, searchIndexQueue, analyticsQueue].forEach(q => {
  orderEventsTopic.addSubscription(new sns.Subscriptions.SqsSubscription(q, {
    rawMessageDelivery: true, // skip SNS envelope – message body only to SQS
    filterPolicy: {}, // no filter – all events to all queues
  }));
// Grant SNS permission to write to SQS (needed for cross-service)
q.addToResourcePolicy(new iam.PolicyStatement({
  principals: [new iam.ServicePrincipal("sns.amazonaws.com")],
  actions: ["sqs:SendMessage"],
  resources: [q.queueArn],
  conditions: { ArnEquals: { "aws:SourceArn": orderEventsTopic.topicArn } },
}));
});

```

SNS Message Filtering — Reduce Subscriber Queue Volume

```

// Email queue: only ORDER_PLACED and ORDER_SHIPPED events (skip INVENTORY_*)
orderEventsTopic.addSubscription(new sns.Subscriptions.SqsSubscription(emailQueue, {
  rawMessageDelivery: true,
  filterPolicy: {
    eventType: sns.SubscriptionFilter.stringFilter({
      allowlist: ["ORDER_PLACED", "ORDER_SHIPPED", "ORDER_CANCELLED"],
    }),
  },
}));

```

Subscriber Queue	SNS Filter	Message Volume	Processing Rate
order-fulfillment.fifo	No filter (all events)	50K/day	Batch=1, visibility=5min, Lambda reserved=5
email-dispatch	ORDER_PLACED + ORDER_SHIPPED only (40% of events)	20K/day	Batch=10, visibility=30s, Lambda reserved=2
search-index	PRODUCT_UPDATED + ORDER_150MAY30ED (30% of events)	150MAY30/day	Batch=10, visibility=30s, Lambda reserved=2
analytics-kinesis	No filter (all events)	50K/day — forwarded to Kinesis Lambda reserved=5, streams to S3 Parquet	

■ Use SNS message filtering to reduce the volume of messages delivered to each SQS subscription. Without filtering, every subscriber receives every message — email-dispatch would receive INVENTORY_RESERVED events it ignores. With filtering, email-dispatch only receives ORDER_PLACED and ORDER_SHIPPED. This reduces email Lambda invocations by 60% and SQS storage costs proportionally.

10-04

EventBridge Custom Bus — Microservice Events

EventBridge custom event buses decouple microservices — order-svc publishes events without knowing who consumes them. EventBridge routes events based on content-based rules to Lambda, SQS, or other targets. ShopFlow uses a custom bus for all domain events between services.

Custom Event Bus CDK

```
const shopflowBus = new events.EventBus(this, "ShopFlowBus", {
  eventBusName: "shopflow-events",
});
// Rule: order placed → trigger inventory Lambda
new events.Rule(this, "OrderPlacedToInventory", {
  eventBus: shopflowBus,
  ruleName: "shopflow-order-placed-inventory",
  description: "Route ORDER_PLACED to inventory reservation",
  eventPattern: {
    source: ["shopflow.order-svc"],
    detailType: ["OrderPlaced"],
    detail: {
      status: ["PENDING"],
      totalAmt: [{ numeric: [">=", 0] }], // all orders
    },
  },
  targets: [new targets.LambdaFunction(inventoryReserverFn, {
    retryAttempts: 3,
    deadLetterQueue: inventoryDlq,
  })],
});
```

Publish Event from order-svc — Java

```
eventBridgeClient.putEvents(PutEventsRequest.builder()
  .entries(PutEventsRequestEntry.builder()
    .eventBusName("shopflow-events")
    .source("shopflow.order-svc")
    .detailType("OrderPlaced")
    .detail(json.writeValueAsString(Map.of(
      "orderId", orderId,
      "userId", userId,
      "totalAmt", total,
      "status", "PENDING",
      "items", items)))
    .build())
  .build());
```

EventBridge vs SNS	EventBridge Custom Bus	SNS Topic
Content-based routing	Yes — route by any JSON field in detail	Filter by message attributes only (limited)
Schema Registry	Yes — auto-discovers event schema	No
Cross-account delivery	Yes — resource policy on bus	Yes — but SNS cross-account is more complex
Throughput	10M events/s per bus (soft)	100K msg/s per topic
Event replay	Yes — archive + replay any event up to 90 days	No
Target types	20+ targets (Lambda, SQS, ECS, Step Functions, Lambda)	Lambda, SQS, HTTP, email, SMS, mobile push
Cost	\$1.00 per M events published	\$0.50 per M SNS deliveries

10-05

EventBridge Pipes — Direct Service-to-Service Integration

EventBridge Pipes connects a source (SQS, DynamoDB Streams, Kinesis) directly to a target (Lambda, SQS, EventBridge bus) with optional filtering and enrichment steps. ShopFlow uses Pipes to stream DynamoDB order events to the EventBridge custom bus without a Lambda intermediary.

EventBridge Pipe CDK — DynamoDB Streams to EventBridge

```
new pipes.CfnPipe(this,"OrderStreamToEventBridge",{
  name: "shopflow-order-stream-pipe",
  roleArn: pipeRole.roleArn,
  source: orderEventsTable.tableStreamArn!,
  sourceParameters: {
    dynamoDbStreamParameters: {
      startingPosition: "LATEST",
      batchSize: 10,
      maximumBatchingWindowInSeconds: 5,
      // Filter: only process INSERT events (new orders)
      filterCriteria: {
        filters: [{ pattern: JSON.stringify({
          eventName: ["INSERT"],
          dynamodb: { NewImage: { eventType: { S: ["ORDER_PLACED"] } } }
        })}],
      },
    },
    target: shopflowBus.eventBusArn,
    targetParameters: {
      eventBridgeEventBusParameters: {
        source: "shopflow.ddb-streams",
        detailType: "OrderPlaced",
      },
    },
    // Optional enrichment: Lambda to transform DDB Stream format to domain event
    enrichment: enrichmentLambda.functionArn,
  });
```

Architecture	Components	Latency	Cost
Without Pipes (old)	DDB Streams → Lambda → EventBridge.putEvents	~500ms Lambda init + 50ms processing	Lambda invocation + EventBridge \$1/M
With Pipes (ShopFlow)	DDB Streams → EventBridge Pipe → EventBridge.putEvents	~50ms (no Lambda hop unless EventBridge Pipes)	EventBridge Pipes \$0.40/M + EventBridge \$

■ EventBridge Pipes filter expressions use the same pattern syntax as EventBridge rules. Test filter patterns using the EventBridge console "Sandbox" before deploying — paste a sample DynamoDB Stream record and verify which records pass. Incorrect filters silently drop events. Enable CloudWatch metrics for the pipe (FilteredMessageCount) to verify the expected number of events are passing through the filter.

Lambda SQS event source mapping reads messages in batches. On partial batch failure, all messages in the batch are retried — including successfully-processed ones. `ReportBatchItemFailures` allows Lambda to return only failed message IDs, preventing successful messages from being reprocessed.

Email Dispatch Lambda with Partial Batch Failure Reporting

```
@Component
public class EmailDispatchHandler
implements RequestHandler<SQSEvent, SQSBatchResponse> {
    private final EmailService emailService;

    @Override
    public SQSBatchResponse handleRequest(SQSEvent event, Context context) {
        List<SQSBatchResponse.BatchItemFailure> failures = new ArrayList<>();
        for (SQSEvent.SQSMessage msg : event.getRecords()) {
            try {
                OrderEvent orderEvent = json.readValue(msg.getBody(), OrderEvent.class);
                emailService.sendOrderConfirmation(orderEvent);
                // SUCCESS: do NOT add to failures list → message deleted from SQS
            } catch (TransientException e) {
                // TRANSIENT failure: return message ID → SQS will redeliver
                failures.add(SQSBatchResponse.BatchItemFailure.builder()
                    .withItemIdentifier(msg.getMessageId()).build());
                log.warn("Transient failure for {}, will retry: {}", msg.getMessageId(), e.getMessage());
            } catch (PermanentException e) {
                // PERMANENT failure (bad email address): log but do NOT retry
                // Message will be deleted (not in failures list) – avoid DLQ spam
                log.error("Permanent failure for {}: {}", msg.getMessageId(), e.getMessage());
                deadLetterService.record(msg, e); // custom tracking
            }
        }
        return SQSBatchResponse.builder().withBatchItemFailures(failures).build();
    }
}
```

SQS Lambda ESM CDK

```
emailLambda.addEventSource(new SqsEventSource(emailQueue, {
    batchSize: 10,
    maxBatchingWindow: Duration.seconds(5), // wait 5s to fill batch
    reportBatchItemFailures: true, // enable partial batch failure reporting
    maxConcurrency: 5, // limit Lambda parallelism from this queue
}));
```

■ Without `reportBatchItemFailures`, any single failure in a batch of 10 causes all 10 messages to be retried. A permanent failure (bad email address) will cycle through `maxReceiveCount` retries for all 10 messages before they land in DLQ — 30 redundant email attempts for 9 valid messages. Always enable `reportBatchItemFailures` for SQS Lambda triggers and distinguish transient from permanent failures.

SNS guarantees delivery to subscribed SQS queues (SQS is synchronous from SNS perspective). For Lambda subscriptions, delivery is asynchronous — SNS has its own DLQ for Lambda subscribers. ShopFlow uses SQS subscription (not Lambda) for all SNS fans-out, so SQS DLQ handles failures.

SNS Subscription Type	Delivery Guarantee	DLQ Support	ShopFlow Usage
SQS	Durable — message stored in SQS until consumed	SQS DLQ	All ShopFlow subscribers (preferred)
Lambda (direct)	Best-effort — retry 3× then DLQ	Lambda DLQ (separate config)	Not used — SQS intermediate preferred

SNS Subscription Type	Delivery Guarantee	DLQ Support	ShopFlow Usage
HTTP/HTTPS endpoint	Best-effort — retry with backoff	HTTP delivery DLQ	External webhooks (partner integrations)
Email/SMS	Best-effort	No DLQ	Ops alerts only (not business events)
Kinesis Firehose	Durable — buffered in Firehose	Firehose error bucket	Analytics stream (high-volume events)

SNS → Lambda DLQ Config (when using direct Lambda subscription)

```
// Direct Lambda subscription: SNS retries 3× then sends to Lambda DLQ
orderEventsTopic.addSubscription(
  new snsSubscriptions.LambdaSubscription(inventoryFn, {
    deadLetterQueue: inventoryDlq,
    filterPolicy: {
      eventType: sns.SubscriptionFilter.stringFilter({
        allowlist: ["ORDER_PLACED"],
      }),
    },
  })
);

// SNS retry policy for HTTP endpoints
// Backoff phases:
// 1. Immediate: 0-3s (3 retries)
// 2. Pre-backoff: 1s intervals (3 retries)
// 3. Backoff: exponential 1s→512s (10 retries)
// 4. Post-backoff: 512s intervals (100,000 retries over 23 days)
```

■ SNS to SQS delivery is synchronous from the SNS publish perspective. When SNS publishes to SQS, it calls the SQS SendMessage API and receives a success/failure response. If SQS is unavailable (extremely rare — >99.99% SLA), SNS retries with exponential backoff. This is why SNS+SQS is more reliable than direct SNS→Lambda: SQS provides the durable buffer that absorbs any transient delivery failures.

10-08

EventBridge Archive & Replay

EventBridge Archive stores all events matching a pattern for up to 90 days. Replay re-delivers archived events to the bus — triggering all matching rules as if the events just occurred. ShopFlow uses archive+replay for incident recovery: replaying missed events after a downstream service outage.

Archive CDK

```
const orderArchive = new events.Archive(this,"OrderEventArchive",{
  archiveName: "shopflow-order-events-archive",
  sourceEventBus: shopflowBus,
  eventPattern: {
    source: ["shopflow.order-svc", "shopflow.ddb-streams"],
  },
  retention: Duration.days(30),
  description: "Archive all order events for 30-day replay capability",
});
```

Replay — Recover from Inventory Service Outage

```
// Scenario: inventory-svc was down for 2 hours (10am-12pm)
// 5,000 ORDER_PLACED events were received but inventory not reserved
// Solution: replay those 2 hours from archive
eventBridgeClient.startReplay(StartReplayRequest.builder()
  .replayName("inventory-recovery-2024-11-29")
  .eventSourceArn(archive.archiveArn())
  .eventStartTime(Instant.parse("2024-11-29T10:00:00Z"))
  .eventEndTime(Instant.parse("2024-11-29T12:00:00Z"))
  .destination(ReplayDestination.builder()
    .arn(shopflowBus.eventBusArn())
```

```
// Optionally limit to specific rules (avoid re-sending emails during replay):
.filterArns(List.of(inventoryRuleArn)) // only inventory rule receives replayed events
.build()
.build();
```

Replay Consideration	Risk	Mitigation
Email service receives replayed events	Customers receive duplicate order confirmation emails	Replay to specific rules only (filterArns). Exclude email-dispatch rule.
Idempotency of consumers	Inventory reserved twice if replay replays already processed event	Already processed event Expression="attribute_not_exists(reservationId)" on inventory reservation
Replay speed	EventBridge replays at up to 10× real-time rate	Rate limit replay via EventBridge replay controls. Monitor consumer DLQ.
Replay order	Replayed events maintain original timestamp	Original createdAt preserved — FIFO consumers see correct ordering

■ Always build idempotent consumers before enabling EventBridge Archive. Replay is only useful if replaying an event a second time is safe. Design inventory reservation with: "if reservation already exists for this orderId, return success without creating a duplicate." Use DynamoDB ConditionExpression or PostgreSQL ON CONFLICT DO NOTHING for idempotent upserts.

EventBridge Schema Registry auto-discovers event schemas from all events on a bus with discovery enabled. It generates downloadable code bindings (Java, Python, TypeScript) for type-safe event deserialization. ShopFlow uses the registry to enforce event contracts across teams.

Enable Schema Discovery

```
// CDK: enable schema discovery on custom bus
const registry = new eventschemas.CfnDiscoverer(this, "SchemaDiscoverer", {
  sourceArn: shopflowBus.eventBusArn,
  description: "Auto-discover ShopFlow event schemas",
});

// Discovered schema – auto-generated JSON Schema for OrderPlaced
// {
//   "$schema": "https://json-schema.org/draft-04/schema#",
//   "title": "shopflow.order-svc@OrderPlaced",
//   "type": "object",
//   "properties": {
//     "version": {"type": "string"},
//     "source": {"type": "string"},
//     "detail-type": {"type": "string"},
//     "detail": {
//       "type": "object",
//       "properties": {
//         "orderId": {"type": "string"},
//         "userId": {"type": "string"},
//         "totalAmt": {"type": "number"},
//         "status": {"type": "string"},
//         "items": {"type": "array"}
//       }
//     }
//   }
// }
```

Schema Registry Feature	Benefit	ShopFlow Use
Auto-discovery	Schemas auto-generated from live events (no manual definition)	OrderPlaced event auto-discovered on first publish
Code bindings	Download Java/Python/TS event POJO auto-generated	shopflow-svc imports OrderPlacedEvent.java — type-safe deserialization
Schema versioning	Track schema evolution; breaking change detection	V1 to V2 of OrderPlaced: added discountAmount field (backward-compatible)
OpenAPI export	Export schema as OpenAPI spec for documentation	API contract published to internal dev portal

SQS monitoring: `ApproximateNumberOfMessagesVisible` (queue depth), `ApproximateAgeOfOldestMessage` (processing lag), `NumberOfMessagesSent`. SNS: `NumberOfMessagesPublished`, `NumberOfNotificationsFailed`. Alert on queue depth growing — indicates consumer is falling behind.

Metric	Service	Alert Threshold	Action Required
<code>ApproximateNumberOfMessagesVisible</code>	SQS	>1000 for 5min (email queue)	Email Lambda throttled — increase reserved concurrency
<code>ApproximateAgeOfOldestMessage</code>	SQS	>300s (fulfillment queue)	Fulfillment Lambda failing — check error logs + DLQ
<code>NumberOfMessagesSent (DLQ)</code>	SQS (DLQ)	>0 any message	Application error — investigate immediately
<code>NumberOfNotificationsFailed</code>	SNS	Any non-zero	Subscriber unreachable — check SQS queue policy, Lambda permissions
<code>NumberOfMessagesPublished</code>	SNS	<10/min during business hours	Order-svc not publishing — check ECS logs

CloudWatch Alarm — SQS Queue Depth CDK

```
new cloudwatch.Alarm(this, "FulfillmentQueueDepth", {
  metric: fulfillmentQueue.metricApproximateNumberOfMessagesVisible({
    period: Duration.minutes(5),
    statistic: "Maximum",
  }),
  threshold: 1000,
  evaluationPeriods: 3,
  datapointsToAlarm: 2,
  alarmDescription: "Fulfillment queue backing up – check Lambda logs",
  comparisonOperator: cloudwatch.ComparisonOperator.GREATER_THAN_THRESHOLD,
}).addAlarmAction(new cwa.SnsAction(opsAlertTopic));
```

■ `ApproximateAgeOfOldestMessage` is a better SLA metric than queue depth. Queue depth can grow if message size is small and Lambda is fast — harmless. Age of oldest message measures processing delay: if the oldest message is 5 minutes old, messages are waiting 5 minutes before processing. Alert when age exceeds your SLA (order fulfillment: 2 minutes). CloudWatch metric: `AWS/SQS` namespace.

EventBridge Scheduler supports flexible time windows, customer-managed KMS encryption, and start/end date for temporary schedules. ShopFlow uses Scheduler for Black Friday pre-warming (temporary schedule), hourly cache refresh, and weekly catalog exports.

Schedule Type	Expression	ShopFlow Job	Notes
Cron (UTC)	<code>cron(0 6 * * ? *)</code>	Daily sales report at 6am UTC	? = any day-of-week when day-of-month specified
Rate	<code>rate(1 hour)</code>	Cache warmer — top 100 products to Redis	Simple interval; no DST adjustment needed
One-time (at)	<code>at(2024-11-28T07:00:00)</code>	BF pre-warm Lambda (one-time)	Auto-deleted after execution
Business hours	<code>cron(0 9-18 ? * MON-FRI *)</code>	Admin digest emails (9am-6pm weekdays)	Cron schedule with hour range
Monthly	<code>cron(0 2 1 * ? *)</code>	Monthly invoice generation (1st at 2am)	1st of every month

Temporary Black Friday Pre-Warm Schedule CDK

```
const bfPreWarm = new scheduler.Schedule(this, "BFPreWarm", {
  scheduleName: "shopflow-bf-prewarm-2024",
  schedule: scheduler.ScheduleExpression.at(
    new Date("2024-11-28T07:00:00Z")), // one-time, auto-deleted after run
  target: new targets.LambdaInvoke(lambdaPreWarmerFn, {
    input: scheduler.ScheduleTargetInput.fromObject({
      action: "PREWARM_PRODUCT_CATALOG",
    })
  })
});
```

```
topProductCount: 1000,
}),
retryPolicy: {
maximumRetryAttempts: 2,
maximumEventAgeInSeconds: 300, // discard if 5min old – pre-warm window
},
}),
enabled: true,
startDate: new Date("2024-11-28T06:55:00Z"), // activate 5min before
endDate: new Date("2024-11-28T07:10:00Z"), // deactivate 10min after
description: "One-time Black Friday 2024 Lambda pre-warm",
});
```

■ EventBridge Scheduler has a soft limit of 1,000,000 schedules per account per region. For per-user reminder schedules (e.g., "remind me in 30 minutes"), this limit is sufficient for ShopFlow's 1M MAU with typical 0.01% conversion to reminders. Use DynamoDB TTL + Streams as an alternative for high-volume per-user schedules that would exceed the Scheduler limit.

10-12

SQS Cost Optimization — Batching & Long Polling

SQS pricing: \$0.40 per million requests (Standard), \$0.50 per million (FIFO). Each API call (SendMessage, ReceiveMessage, DeleteMessage) counts as one request. Batching reduces API calls by up to 10×. Long polling eliminates empty receive calls. ShopFlow saves ~\$50/month with these optimizations.

Optimization	Mechanism	Request Reduction	Monthly Saving (ShopFlow)
SendMessageBatch	Send up to 10 msgs per API call	10× fewer SendMessage API calls	200K orders/day ÷ 10 = 20K vs 200K calls/day
ReceiveMessageBatch	Receive up to 10 msgs per ReceiveMessage call	10× fewer ReceiveMessage calls	Lambda ESM already batches by default
DeleteMessageBatch	Delete up to 10 msgs per call	10× fewer DeleteMessage calls	Automatic with Lambda ESM + reportBatchItemFailures
Long polling (WaitTimeSeconds=10)	Wait 10s for 10 messages vs empty response	Eliminates empty receive calls	Lambda ESM uses long polling automatically
SQS + Lambda ESM	Lambda ESM manages polling, batching	No polling cost from application	All SQS costs included in Lambda invocation model

Manual Batch Send — Java

```
// Send 10 order events in one BatchSendMessage call
List<SendMessageBatchRequestEntry> entries = orders.stream()
    .map(o -> SendMessageBatchRequestEntry.builder()
        .id(o.getOrderID()) // unique ID within this batch request
        .messageBody(json.writeValueAsString(o))
        .messageGroupId(o.getOrderID()) // FIFO: one group per order
        .messageDeduplicationId(o.getOrderID() + "#" + System.currentTimeMillis())
        .build())
    .collect(toList());
// Must send max 10 per batch – SQS limit
Lists.partition(entries, 10).forEach(batch ->
    sqsClient.sendMessageBatch(SendMessageBatchRequest.builder()
        .queueUrl(fulfillmentQueueUrl)
        .entries(batch)
        .build()));
```

■ SQS Free Tier: 1 million requests per month (forever, not just first year). ShopFlow email queue: 200K messages/day = 6M/month × 3 SQS calls (send+receive+delete) = 18M/month → only 17M requests billed after free tier. At \$0.40/M: \$6.80/month for email queue. SQS is one of the cheapest AWS services per message delivered — almost always preferable to running a self-managed message broker (RabbitMQ, Kafka on EC2).

10-13

Security — Encryption, IAM & VPC Endpoints

SQS and SNS support server-side encryption with AWS KMS. IAM policies control which principals can send/receive/delete messages. VPC endpoints eliminate NAT Gateway costs for ECS task communication with SQS/SNS. Resource-based policies on queues restrict cross-account access.

Security Control	SQS / SNS	ShopFlow Config	Threat Mitigated
SSE-SQS (free)	SQS only	Standard queues: SSE-SQS (AES-256, no KMS cost)	Only at rest — no KMS cost
SSE-KMS (CMK)	SQS + SNS	FIFO order-fulfillment queue: CMK; email queue: SSE-SQS	Regulatory compliance, audit trail, key rotation
IAM send policy	SQS + SNS	ECS task role: sqs:SendMessage on specific queue ARN	Prevent unauthorized message injection
IAM receive policy	SQS	Lambda execution role: sqs:ReceiveMessage + sqs:DeleteMessage	Prevent unauthorized queue draining
SNS resource policy	SNS	Allow sqs:SendMessage from sns.amazonaws.com (source SNS topic)	Prevent SNS topic hijacking
VPC endpoint	SQS + SNS	Interface endpoints in app-tier subnet	No NAT charge; SQS/SNS traffic stays in AWS

SQS Resource Policy — Restrict to VPC Endpoint

```
emailQueue.addToResourcePolicy(new iam.PolicyStatement({
  sid: "DenyNonVpcAccess",
  effect: iam.Effect.DENY,
  principals: [new iam.AnyPrincipal()],
  actions: ["sqs:SendMessage", "sqs:ReceiveMessage"],
  resources: [emailQueue.queueArn],
  conditions: {
    StringNotEquals: {
      "aws:sourceVpce": sqsVpcEndpoint.vpcEndpointId,
    },
  },
}));
```

■ SQS and SNS VPC Interface endpoints cost \$0.01/hour/AZ + \$0.01/GB data. ShopFlow VPC: 3 AZs × 2 endpoints (SQS + SNS) = 6 interface endpoints × \$0.01/hr × 720hr/mo = \$43.20/mo. Without endpoints, SQS/SNS traffic exits via NAT Gateway at \$0.045/GB. At 10GB/day SNS+SQS traffic: NAT cost = \$0.045 × 10 × 30 = \$13.50/mo. VPC endpoints are MORE expensive than NAT for low-volume traffic — evaluate break-even before adding endpoints.

10-14

SNS Mobile Push & SMS for Order Notifications

SNS supports push notifications to iOS (APNs), Android (FCM), and SMS. ShopFlow uses SNS Mobile Push for order status updates on the ShopFlow mobile app and SMS for high-priority shipping alerts. SNS handles platform credential management and scales to millions of devices.

Channel	SNS Platform	ShopFlow Event	Volume
iOS push (APNs)	AWS/APNS	ORDER_SHIPPED → push to iOS app	50K/day
Android push (FCM)	AWS/GCM	ORDER_SHIPPED → push to Android app	100K/day
SMS	AWS/SMS	ORDER_DELAYED (SLA breach) alert only	500/day (selective)
Email (SES via SNS)	Not direct — SNS → SES	ORDER_PLACED SES confirmation	200K/day

SNS Mobile Push CDK + Java SDK

```
// CDK: create Platform Application (FCM/GCM)
const fcmApp = new sns.CfnPlatformApplication(this,"FcmApp",{
  name: "shopflow-android",
  platform: "GCM",
  attributes: {
    PlatformCredential: fcmServerKeySecret.secretValue.unsafeUnwrap(),
  },
});
// Register device token (on mobile app install/launch)
```

```

CreatePlatformEndpointResponse endpointResp = snsClient.createPlatformEndpoint(
    CreatePlatformEndpointRequest.builder()
        .platformApplicationArn(fcmAppArn)
        .token(fcmDeviceToken) // from Firebase SDK on device
        .customUserData(userId) // link to ShopFlow user
        .build());
String endpointArn = endpointResp.endpointArn(); // persist in Aurora per userId
// Send push notification
snsClient.publish(PublishRequest.builder()
    .targetArn(endpointArn)
    .messageStructure("json")
    .message(json.writeValueAsString(Map.of(
        "GCM", json.writeValueAsString(Map.of(
            "notification", Map.of(
                "title", "Your order has shipped!",
                "body", "Order " + orderId + " is on its way"),
                "data", Map.of("orderId", orderId, "type", "ORDER_SHIPPED"))))))))
    .build());

```

■ SNS device endpoint tokens expire when users uninstall and reinstall the app or rotate FCM/APNs tokens. Subscribe to SNS EndpointDisabled events via CloudWatch Events to detect expired endpoints. When an endpoint is disabled, update your Aurora user record to mark the endpoint invalid. Without cleanup, you accumulate disabled endpoints that slow down fan-out and incur SNS charges for failed deliveries.

10-15

Cost Analysis — SQS + SNS + EventBridge Total

Full monthly messaging cost for ShopFlow production: SQS Standard queues, SQS FIFO, SNS fan-out, EventBridge custom bus events, and EventBridge Scheduler invocations. Messaging infrastructure costs under \$30/month — significantly cheaper than running self-managed Kafka or RabbitMQ on EC2.

Component	Monthly Volume	Unit Price	Monthly Cost
SQS Standard (email, search, analytics)	1.8M requests (3 queues)	\$0.40/M (after 1M free)	\$6.80
SQS FIFO (order-fulfillment.fifo)	1.5M requests	\$0.50/M (after 1M free)	\$0.25
SNS fan-out deliveries to SQS	2M deliveries	\$0.50/M	\$1.00
SNS Mobile Push (FCM + APNs)	4.5M push/month	\$0.50/M (first 1M free)	\$1.75
EventBridge custom bus (order events)	2M events/month	\$1.00/M (first 1M free)	\$2.00
EventBridge Scheduler	30K invocations/month	\$1.00/M (first 14M free)	\$0
SQS + SNS VPC Interface endpoints	6 endpoints × 720hr	\$0.01/hr/endpoint	\$43.20
Total (without VPC endpoints)			\$11.80/month
Total (with VPC endpoints)			\$55.00/month

Self-Managed vs Managed	Kafka on EC2 (3-broker)	SQS + SNS + EventBridge
Instance cost	3× r5.xlarge = \$0.252/hr × 720 = \$545/mo	\$0 (serverless)
Storage (EBS for logs)	3× 500GB gp3 = \$0.08/GB × 1.5TB = \$120/mo	\$0
Ops (patching, monitoring, scaling)	>4 hours/month engineering time	\$0 (fully managed)
Total monthly cost	\$665+ (plus eng time)	\$11.80-55.00 (predictable)
Max throughput	Scales to millions/s with tuning	SQS: 300K msg/s Standard; SNS: 300K/s

■ Managed messaging (SQS + SNS + EventBridge) is almost always cheaper than self-managed Kafka for workloads under 50,000 messages/second. Kafka on EC2 breaks even at high sustained throughput (>50K msg/s sustained for >12 hours/day) or when Kafka-specific features (log compaction, consumer group rebalancing, stream processing with Kafka Streams) are required. ShopFlow at 50K messages/day is nowhere near that threshold.

SQS maximum message size is 256 KB. Order events with large item arrays exceed this. The SQS Extended Client Library transparently offloads payloads to S3, storing only the S3 pointer in SQS. Consumers transparently retrieve from S3 before processing — no application-layer changes needed.

Extended Client Setup — Java

```
ExtendedClientConfiguration extConfig = new ExtendedClientConfiguration()
    .withPayloadSupportEnabled(s3Client, "shopflow-sqs-large-payloads")
    .withPayloadSizeThreshold(100 * 1024) // offload payloads > 100KB to S3
    .withAlwaysThroughS3(false); // only offload when > threshold
SqsClient sqsExtendedClient = new AmazonSQSExtendedClient(
    SqsClient.builder().build(), extConfig);
// Send: if body > 100KB → writes to S3, sends S3 pointer to SQS
sqsExtendedClient.sendMessage(SendMessageRequest.builder()
    .queueUrl(largeOrderQueueUrl)
    .messageBody(json.writeValueAsString(largeOrderEvent))
    .build());
// Consumer: receive from SQS; Extended Client auto-fetches S3 payload
// Delete: removes SQS message; optionally auto-deletes S3 object
```

Scenario	SQS Only	SQS Extended Client (S3 offload)
Message size	Max 256 KB hard limit	Up to 2 GB (S3 object limit)
Order with 500 line items (~200KB)	Rejected by SQS	Stored in S3; SQS msg = S3 pointer (300B)
Latency overhead	None	~50ms extra S3 GetObject on consumer read
Cost	\$0.40/M SQS requests	\$0.40/M SQS + \$0.023/GB S3 + GET requests
Cleanup	SQS auto-deletes	Must delete S3 object (lifecycle rule as safety net)

■ Set an S3 lifecycle rule on the large-payloads bucket to expire objects after 14 days (matching SQS max retention). Without this, processed messages leave orphaned S3 objects that accumulate indefinitely. Configure `withCleanupS3Payload(true)` in the Extended Client as a first line of defense, with the lifecycle rule as a safety net for messages that are never consumed.

ShopFlow Phase 2 integrates with a 3PL (third-party logistics) warehouse in a separate AWS account. Cross-account SNS subscription delivers ORDER_SHIPPED events to the 3PL SQS queue. The 3PL account adds a resource policy allowing ShopFlow SNS; ShopFlow adds the 3PL SQS ARN as an SNS subscriber.

Cross-Account SQS Resource Policy — 3PL Account

```
// Step 1: In 3PL account – allow ShopFlow SNS to deliver
thirdPartyLogisticsQueue.addToResourcePolicy(new iam.PolicyStatement({
    sid: "AllowShopFlowSNSDelivery",
    effect: iam.Effect.ALLOW,
    principals: [new iam.ServicePrincipal("sns.amazonaws.com")],
    actions: ["sqs:SendMessage"],
    resources: [thirdPartyLogisticsQueue.queueArn],
    conditions: {
        ArnLike: {
            "aws:SourceArn": "arn:aws:sns:us-east-1:111122223333:shopflow-order-events",
        },
        StringEquals: {
            "aws:SourceAccount": "111122223333", // ShopFlow AWS account ID
        },
    },
}));
```

```
});
```

ShopFlow Account — Add 3PL SQS as SNS Subscriber

```
// Step 2: ShopFlow account – subscribe 3PL SQS ARN to SNS topic
const thirdPartyQueue = sqs.Queue.fromQueueArn(this, "ThirdPartyQueue",
"arn:aws:sqs:us-east-1:444455556666:shopflow-3pl-orders");
orderEventsTopic.addSubscription(
new snsSubscriptions.SqsSubscription(thirdPartyQueue, {
rawMessageDelivery: true,
filterPolicy: {
eventType: sns.SubscriptionFilter.stringFilter({
allowlist: ["ORDER_PLACED", "ORDER_SHIPPED", "ORDER_CANCELLED"],
}),
},
})
);
```

Pattern	Mechanism	Key Requirement	Security Control
Cross-account SNS to SQS	SQS in ShopFlow acct; SQS resource policy on 3PL queue	Policy on 3PL queue confirms subscription URL sent to SQS	aws:SourceAccount prevents confused deputy att
Cross-account EventBridge	Bridge target = event bus ARN in partner account	Resource policy on target bus allowing first-party account	Trust policy on target bus; no wildcard principals
Cross-region EventBridge	Global endpoint + Route 53 health-check	Two event buses (primary + secondary) in same region	Same IAM as single-region; global endpoint route

■ Cross-account SNS to SQS requires `aws:SourceAccount` condition to prevent confused deputy: without it, any SNS topic in any AWS account could deliver to the 3PL queue. With `aws:SourceAccount=111122223333`, only ShopFlow SNS topics deliver — even if `SourceArn` is spoofed. Always pair `ArnLike SourceArn` with `StringEquals SourceAccount` for cross-account SNS subscriptions.

10-BP

Best Practices & Anti-Patterns

- ✓ Use SNS + SQS fan-out for all order events — decouple producers from consumers completely
- ✓ Configure DLQ on every SQS queue — never lose messages silently; alert on DLQ depth >0
- ✓ Enable `reportBatchItemFailures` for SQS Lambda triggers — prevent retry flooding on one failure
- ✓ Use SQS FIFO with `MessageGroupId` per order — guarantee per-order processing sequence
- ✓ Set visibility timeout to 6× Lambda timeout — prevent duplicate processing on slow invocations
- ✓ Use SNS message filtering to reduce subscriber queue volume — 40-60% reduction in unwanted messages
- ✓ Use EventBridge Archive + Replay for all domain events — enables incident recovery without data loss
- ✓ Build idempotent consumers for all event-driven handlers — safe for EventBridge replay and SQS retry
- ✓ Use EventBridge custom bus for microservice events — decouple source from all consumers
- ✓ Use long polling (`WaitTimeSeconds=20`) for all SQS consumers — eliminate empty receive costs
- ✓ Use `SendMessageBatch` (10 msgs per call) for high-volume producers — 10× fewer API calls
- ✓ Enable EventBridge Schema Registry on custom bus — enforce event contracts across teams
- ✓ Use EventBridge Pipes to connect DDB Streams to EventBridge without Lambda intermediary
- ✓ Monitor `ApproximateAgeOfOldestMessage` — better SLA metric than queue depth
- ✓ Use SSE-KMS on FIFO queues for PCI compliance; SSE-SQS on standard queues (free, simpler)
- Never use SQS as a database — messages are not queryable; maximum retention is 14 days
- Do not use SNS email subscriptions for business events — use SES directly for transactional emails
- Avoid very large messages (>256KB) — SQS limit is 256KB; use S3 pointer pattern for large payloads
- Do not ignore DLQ messages — DLQ depth >0 means messages are being permanently failed; always investigate
- Do not use SNS Pub/Sub for ordered delivery — SNS is unordered; use SQS FIFO for ordered processing

■ Avoid tightly coupling Lambda timeout and SQS visibility timeout — Lambda timeout changes break visibility timeout SLA

10-QR

Quick Reference & Interview Q&A

Concept	Key Value / Rule
SQS max message size	256 KB. For larger payloads: store in S3, put S3 key in message (S3 extended client library).
SQS FIFO throughput	300 msg/s (no batching), 3,000 msg/s (with batch API). Enable high-throughput mode for higher limits.
SQS visibility timeout	Time message is hidden after ReceiveMessage. Set to 6× Lambda timeout. Default: 30s. Max: 12 hours.
SQS DLQ trigger	Message delivered maxReceiveCount times without delete → moved to DLQ automatically.
SNS fan-out limit	10 million subscriptions per topic. 12.5 million messages/s publish rate per topic.
SNS filter policy	Filter by message attributes (stringFilter, numericFilter). Applied at SNS — reduces SQS delivery volume.
EventBridge rule limit	300 rules per event bus (default). 2,000 targets per rule. 100 buses per account.
EventBridge archive max	90 days retention. Replay at up to 10× real-time. filterArns to limit replay to specific rules.
SQS pricing	\$0.40/M Standard, \$0.50/M FIFO. Free tier: 1M requests/month forever.
EventBridge pricing	\$1.00/M custom events published. Default bus (AWS service events): free.

Top Interview Questions

Question	Answer
SNS vs SQS — when each?	SNS: one-to-many fan-out (broadcast to multiple subscribers simultaneously). SQS: one-to-one durable queue.
How does SQS DLQ work?	Consumer receives message (ReceiveCount increments). If consumer deletes message → success. If consumer fails to delete, message is moved to DLQ.
SQS FIFO exactly-once delivery	Producer sets MessageDeduplicationId (or content-based deduplication). SQS deduplicates within 5-minute deduplication window.
EventBridge vs SNS for microservices	EventBridge: content-based routing (filter by any JSON field), schema registry, archive+replay, 20+ target types. SNS: simple fan-out, no filtering.
How to handle poison-pill messages	Isolate in SQS DLQ. Investigate root cause (malformed payload, unexpected data). Solution: idempotent consumer logic, retries with exponential backoff.